

Project Handbook

Overview

This repository implements an enterprise AI SQL agent platform with a Python backend API, a React frontend, schema-aware SQL generation, MySQL read-only execution, query history, and report export.

Architecture

User request flow:

1. Browser frontend sends a natural-language question.
2. FastAPI backend receives /query and routes to the AI SQL agent.
3. SQL generation uses the knowledge graph, query planner, prompt builder, and Ollama local model.
4. Validated SQL is executed against MySQL and results are returned.

Component Breakdown

Backend

- app/api/main.py: FastAPI application and endpoints.
- app/agent/sql_agent.py: Orchestration pipeline.
- app/agent/query_planner.py: Builds deterministic execution plans.
- app/agent/ambiguity_checker.py: Detects ambiguous user questions.
- app/llm/sql_generator.py: Generates SQL via Ollama or fallback logic.
- app/prompt/prompt_builder.py: Builds strict prompts with table/column context.
- app/database/read_executor.py: Executes read-only MySQL queries.
- app/knowledge/knowledge_graph.py: Loads schema and relationship metadata.
- app/utils/report_generator.py: Exports CSV, Excel, and PDF reports.
- app/sql_history/query_history.py: Logs query history and audits.

Frontend

- React + Vite app in frontend/src to provide query, history, admin, and settings UI.

Tech Stack

- Python, FastAPI, Uvicorn, SQLAlchemy, PyMySQL, pandas, reportlab.
- React, Vite, lucide-react, recharts.
- Ollama local model for offline SQL generation.
- MySQL read-only database backend.

Why this design

- FastAPI for fast API development and static file hosting.
- React + Vite for responsive frontend and rapid development.
- Knowledge graph and deterministic prompt structure to reduce hallucinations.
- Ollama local model to keep inference offline and self-contained.
- SQL validation and EXPLAIN checks for safer query execution.

Setup & Run

1. Create a Python virtual environment and install requirements.
2. Add database credentials to .env: DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD.
3. Start backend: python app/main.py or python -m uvicorn app.api.main:app --reload.
4. Start frontend: cd frontend && npm install && npm run dev.

Useful Endpoints

- POST /query
- POST /query/stream
- GET /history
- GET /admin/status, /admin/schema, /admin/tables, /admin/knowledge-graph

- POST /admin/rebuild-embeddings, /admin/refresh-metadata
- POST /export/csv, /export/excel, /export/pdf, /export/json, /export/sql

Key Files

- requirements.txt, frontend/package.json
- app/api/main.py, app/agent/sql_agent.py, app/llm/sql_generator.py, app/database/read_executor.py.

Notes

- The system is designed for read-only analytics and safe SQL generation.
- Ollama should run locally at <http://127.0.0.1:11434> for LLM mode.
- Generated reports are saved under reports/ and exposed via /reports/.

Quick Start Commands

```
python -m venv .venv
```

```
.venv\Scripts\Activate.ps1
```

```
pip install -r requirements.txt
```

```
python app/main.py
```

```
cd frontend && npm install && npm run dev
```